

Python CSV Module

```
# =====
# CSV MODULE - COMPLETE EXPLANATION IN ONE PYTHON SCRIPT
# =====

import csv
from pathlib import Path

# =====
# 1. CREATE SAMPLE CSV FILE
# =====

file_path = Path("customers.csv")

customers = [
    ["id", "name", "age", "city"],
    [1, "Ahmed", 25, "Cairo"],
    [2, "Mohamed", 30, "Giza"],
    [3, "Omar", 28, "Alexandria"]
]

with file_path.open("w", newline="", encoding="utf-8") as file:
    writer = csv.writer(file)
    writer.writerows(customers)

print("CSV file created!")

# =====
# 2. READ CSV FILE
# =====

with file_path.open("r", newline="", encoding="utf-8") as file:
    reader = csv.reader(file)

    for row in reader:
        print(row)

# =====
# 3. READ CSV ROW BY ROW
# =====

print("\nRows:")

with file_path.open("r", newline="", encoding="utf-8") as file:
    reader = csv.reader(file)

    for row in reader:
        print(
            "ID:", row[0],
            "Name:", row[1],
            "Age:", row[2],
            "City:", row[3]
        )
    )
```

```

# =====
# 4. SKIP HEADER
# =====

print("\nWithout header:")

with file_path.open("r", newline="", encoding="utf-8") as file:
    reader = csv.reader(file)
    next(reader)
    for row in reader:
        print(row)

# =====
# 5. csv.DictReader()
# =====

# DictReader converts each row into a dictionary.

print("\nUsing DictReader:")

with file_path.open("r", newline="", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row)

# Example:

# {
#     "id": "1",
#     "name": "Ahmed",
#     "age": "25",
#     "city": "Cairo"
# }

# =====
# 6. ACCESS DICTIONARY VALUES
# =====

print("\nAccess columns:")

with file_path.open("r", newline="", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(
            row["name"],
            row["city"]
        )

# =====
# 7. CSV DATA TYPES
# =====

print("\nData types:")

with file_path.open("r", newline="", encoding="utf-8") as file:

```

```

reader = csv.DictReader(file)

for row in reader:
    print(
        row["name"],
        type(row["name"])
    )

    print(
        row["age"],
        type(row["age"])
    )

# CSV values are read as strings.

# So:
#
# "25"
#
# is a string, not an integer.

# =====
# 8. CONVERT DATA TYPES
# =====

print("\nConverted types:")

with file_path.open("r", newline="", encoding="utf-8") as file:
    reader = csv.DictReader(file)

    for row in reader:
        customer_id = int(row["id"])
        age = int(row["age"])

        print(
            customer_id,
            type(customer_id),
            age,
            type(age)
        )

# =====
# 9. CREATE CSV USING DictWriter
# =====

output_file = Path("employees.csv")

employees = [
    {
        "id": 1,
        "name": "Ahmed",
        "department": "Data Engineering",
        "salary": 15000
    },
    {
        "id": 2,
        "name": "Mohamed",
        "department": "Analytics",
        "salary": 12000
    },
    {
        "id": 3,
        "name": "Omar",

```

```

        "department": "Data Engineering",
        "salary": 18000
    }
]

fieldnames = [
    "id",
    "name",
    "department",
    "salary"
]

with output_file.open(
    "w",
    newline="",
    encoding="utf-8"
) as file:

    writer = csv.DictWriter(
        file,
        fieldnames=fieldnames
    )

    writer.writeheader()

    writer.writerows(employees)

print("\nEmployees CSV created!")

# =====
# 10. READ CSV USING DictReader
# =====

print("\nEmployees:")

with output_file.open(
    "r",
    newline="",
    encoding="utf-8"
) as file:

    reader = csv.DictReader(file)

    for employee in reader:

        print(
            employee["name"],
            employee["department"],
            employee["salary"]
        )

# =====
# 11. FILTER CSV DATA
# =====

print("\nData Engineering department:")

with output_file.open(
    "r",
    newline="",
    encoding="utf-8"
) as file:

    reader = csv.DictReader(file)

    for employee in reader:

```

```

        if employee["department"] == "Data Engineering":
            print(employee)

# =====
# 12. FILTER BY NUMERIC VALUE
# =====

print("\nEmployees with salary > 15000:")

with output_file.open(
    "r",
    newline="",
    encoding="utf-8"
) as file:

    reader = csv.DictReader(file)

    for employee in reader:

        salary = int(employee["salary"])

        if salary > 15000:

            print(employee)

# =====
# 13. TRANSFORM CSV DATA
# =====

transformed_employees = []

with output_file.open(
    "r",
    newline="",
    encoding="utf-8"
) as file:

    reader = csv.DictReader(file)

    for employee in reader:

        employee["salary"] = int(employee["salary"])

        employee["salary_after_bonus"] = (
            employee["salary"] * 1.10
        )

        transformed_employees.append(employee)

print("\nTransformed data:")

for employee in transformed_employees:

    print(employee)

# =====
# 14. WRITE TRANSFORMED DATA
# =====

transformed_file = Path("employees_transformed.csv")

fieldnames = [
    "id",
    "name",

```

```

        "department",
        "salary",
        "salary_after_bonus"
    ]

    with transformed_file.open(
        "w",
        newline="",
        encoding="utf-8"
    ) as file:

        writer = csv.DictWriter(
            file,
            fieldnames=fieldnames
        )

        writer.writeheader()

        writer.writerows(transformed_employees)

    print("\nTransformed CSV created!")

# =====
# 15. CSV DELIMITER
# =====

semicolon_file = Path("products.csv")

products = [
    ["id", "name", "price"],
    [1, "Laptop", 25000],
    [2, "Phone", 15000],
    [3, "Tablet", 10000]
]

with semicolon_file.open(
    "w",
    newline="",
    encoding="utf-8"
) as file:

    writer = csv.writer(
        file,
        delimiter=";"
    )

    writer.writerows(products)

    print("\nSemicolon CSV created!")

# =====
# 16. READ SEMICOLON CSV
# =====

print("\nSemicolon CSV:")

with semicolon_file.open(
    "r",
    newline="",
    encoding="utf-8"
) as file:

    reader = csv.reader(
        file,
        delimiter=";"
    )

```

```

        for row in reader:

            print(row)

# =====
# 17. CSV QUOTING
# =====

file_path = Path("quotes.csv")

data = [
    ["id", "name", "description"],
    [
        1,
        "Laptop",
        "Laptop, powerful and fast"
    ],
    [
        2,
        "Phone",
        "Phone, lightweight"
    ]
]

with file_path.open(
    "w",
    newline="",
    encoding="utf-8"
) as file:

    writer = csv.writer(file)

    writer.writerows(data)

print("\nCSV with commas inside values created.")

# =====
# 18. HANDLE MISSING VALUES
# =====

missing_file = Path("customers_missing.csv")

data = [
    ["id", "name", "age", "city"],
    [1, "Ahmed", 25, "Cairo"],
    [2, "Mohamed", "", "Giza"],
    [3, "", 28, "Alexandria"]
]

with missing_file.open(
    "w",
    newline="",
    encoding="utf-8"
) as file:

    writer = csv.writer(file)

    writer.writerows(data)

print("\nMissing values:")

with missing_file.open(
    "r",
    newline="",
    encoding="utf-8"
) as file:

```

```

reader = csv.DictReader(file)

for row in reader:
    if not row["name"]:
        print("Missing name:", row)

    if not row["age"]:
        print("Missing age:", row)

# =====
# 19. SIMPLE CSV ETL PIPELINE
# =====

input_file = Path("orders.csv")
output_file = Path("orders_clean.csv")

# -----
# EXTRACT
# -----

orders = [
    ["order_id", "customer", "amount"],
    [1, "Ahmed", 500],
    [2, "Mohamed", 1200],
    [3, "Omar", -100],
    [4, "Youssef", 800]
]

with input_file.open(
    "w",
    newline="",
    encoding="utf-8"
) as file:
    writer = csv.writer(file)
    writer.writerows(orders)

# -----
# EXTRACT
# -----

valid_orders = []

with input_file.open(
    "r",
    newline="",
    encoding="utf-8"
) as file:
    reader = csv.DictReader(file)

    for order in reader:
        order["amount"] = float(order["amount"])
        valid_orders.append(order)

# -----
# TRANSFORM + VALIDATE
# -----

clean_orders = []

```



```

for order in valid_orders:
    # Remove negative amounts
    if order["amount"] < 0:
        continue

    # Add status
    if order["amount"] >= 1000:
        order["status"] = "High Value"
    else:
        order["status"] = "Normal"

    clean_orders.append(order)

# -----
# LOAD
# -----

fieldnames = [
    "order_id",
    "customer",
    "amount",
    "status"
]

with output_file.open(
    "w",
    newline="",
    encoding="utf-8"
) as file:
    writer = csv.DictWriter(
        file,
        fieldnames=fieldnames
    )

    writer.writeheader()

    writer.writerows(clean_orders)

print("\n===== CSV ETL =====")

for order in clean_orders:
    print(order)

print("\nETL completed!")
print("Output:", output_file)

# =====
# 20. MOST IMPORTANT CSV FUNCTIONS
# =====

print("""
=====
CSV MODULE - IMPORTANT
=====

csv.reader()

```

```
    Read CSV rows as lists.

csv.writer()
    Write lists to CSV.

csv.DictReader()
    Read CSV rows as dictionaries.

csv.DictWriter()
    Write dictionaries to CSV.

writer.writerow()
    Write one row.

writer.writerows()
    Write multiple rows.

writer.writeheader()
    Write column names.
```

```
=====
IMPORTANT OPTIONS
=====
```

```
delimiter=","
    Comma-separated CSV.

delimiter=";"
    Semicolon-separated CSV.

newline=""
    Recommended when opening CSV files.

encoding="utf-8"
    Handle text correctly.
```

```
=====
IMPORTANT REMEMBER
=====
```

```
CSV
↓
csv.reader()
↓
list

CSV
↓
csv.DictReader()
↓
dictionary
```

```
Python data
↓
csv.writer()
↓
CSV
```

```
Python dictionaries
↓
csv.DictWriter()
↓
CSV
```

```
=====
```

DATA ENGINEERING

CSV File



EXTRACT



DictReader()



Python Dictionaries



VALIDATE



TRANSFORM



DictWriter()



Clean CSV



LOAD

=====
""")